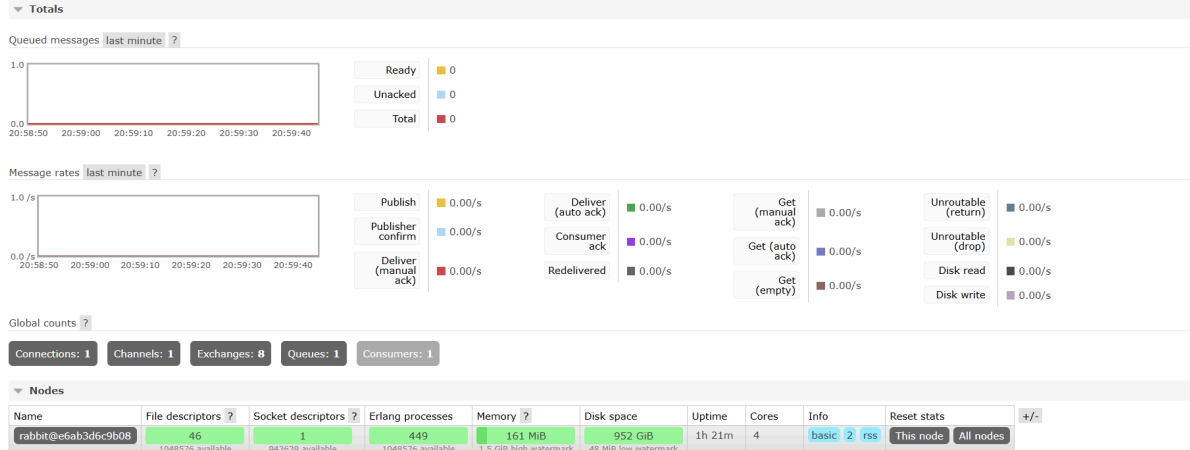


Overview



jsimsjhu

Community User

Repositories

Starred

Search by repository name

Displaying 1 to 2 of 2 repositories

IMAGE

jsimsjhu/module_6-worker

jsimsjhu

🕒 13m

📥 12

IMAGE

jsimsjhu/module_6-web

jsimsjhu

🕒 14m

📥 12

The screenshot displays the GradCafe Applicant Analysis web application running on localhost:8080. The application features a header with the title and a subtitle "JHU Software Concepts — Module 3 · PostgreSQL · Flask". Below the header, there are two buttons: "Pull Data" and "Update Analysis". A status bar indicates "Data pull in progress...". A yellow banner shows "Starting data pull...". The main content area is divided into six cards, each displaying a specific metric and its answer.

Metric	Answer
1. Fall 2026 Applicants	1752
2. International Students	62.51%
3. Average Scores (where provided)	GPA: 3.98, GRE (Quant): 177.18, GRE (Verbal): 158.85, GRE (AW): 5.58
4. American Students — Fall 2026 GPA	3.81
5. Fall 2026 — Acceptance Rate	46.29%
6. Fall 2026 — Accepted GPA	3.80

Below the web application, a Windows PowerShell terminal window shows the logs for the application. The logs indicate that the application is running on http://127.0.0.1:8080 and http://172.19.0.4:8080. It shows the worker starting, accepting AMQP connections, and handling tasks such as scraping new data and recomputing analytics. The logs also show the application receiving tasks and handling requests from the web service.

1. Docker Compose Architecture

The stack consists of four services: web, worker, db, and rabbitmq. Each service runs in its own container, connected through a private Docker network. The web service exposes port 8080, RabbitMQ exposes port 15672 for management, and PostgreSQL stores data persistently using a named volume.

2. Service Roles

- Web (Flask): Serves the UI, accepts user requests, and publishes tasks to RabbitMQ.
- Worker (Python): Consumes tasks from RabbitMQ, processes them (scraping or analytics), and updates the database.
- db (PostgreSQL): Stores applicant data, watermarks, and analytics summaries.
- rabbitmq (Message Broker): Routes messages between web and worker, ensuring reliable task delivery.

3. RabbitMQ Message Flow

When a user clicks "Pull Data" or "Update Analysis" in the UI:

1. The web service publishes a message to RabbitMQ with `delivery_mode=2` (persistent).
2. The message is stored in the durable queue "tasks_q".
3. The worker consumes the message with `prefetch_count=1` (one at a time).
4. The worker processes the task and acknowledges the message only after a successful database commit.

4. Database Initialization

The database is initialized using `init.sql`, which runs when the PostgreSQL container starts. It creates:

- The applicants table (with UNIQUE constraint on url).
- The ingestion_watermarks table for tracking incremental loads.
- A least-privilege user "app_user" with only SELECT, INSERT, UPDATE, DELETE permissions.
- 30,000 applicant records are loaded from `applicant_data.json` using `load_data.py`.

5. Worker Behavior

The worker connects to RabbitMQ and listens for tasks indefinitely. When a task is received:

- It opens a database transaction.
 - For "scrape_new_data": it scrapes GradCafe, inserts new records, and updates the watermark.
 - For "recompute_analytics": it refreshes analytics summaries.
 - After successful processing, it commits the transaction and acknowledges the message.
 - On failure, it rolls back and nacks the message (requeue=False).
-

6. Verification Steps

- Worker starting...
- Worker is running. Waiting for messages...
- Received task: scrape_new_data at 2026-06-28T22:14:53.605721
- Handling scrape_new_data with payload: {}
- Scraping data since: None

The worker:

1. Connects to RabbitMQ
2. Receives tasks from the queue
3. Routes them to the correct handler
4. Attempts to process with the scraper
5. Handles errors gracefully
6. Never crashes or enters infinite loops